Capability models for Go: An analysis of the most popular dependencies

Group 22
Vilhelm Prytz vprytz@kth.se
Filip Dimitrijevic filipdi@kth.se

April 22, 2026

1 Background

A capability is a permission that grants a program access to a particular system resource, such as a file system or operating system. A capability model describes what capabilities a particular piece of software can perform, based on analysis of its code.[1] This is relevant when analyzing open source software dependencies, as it gives insight into what system resources a dependency can access, often without the developer's knowledge. Tools such as Capslock implement this in practice by statically analyzing Go packages to produce such models.[2]

2 Goal of Project

The goal of the project is to analyze the top 50 Golang packages by the number of GitHub stars they have. The analysis will look at capability models for each project using Capslock.[3] Capslock analyzes Golang code and looks at what capabilities a program **actually** uses. How have the capabilities as modeled by our capabilities model of each version changed over time, and why?

In addition, we also thought about analyzing over-capability in the packages, but decided not to include this to keep the scope of the project reasonable.

3 Relevance to Language-Based Security

Capability models are relevant to language-based security as they provide a practical way to reason about and enforce the principle of least privilege in software. By analyzing what system resources a dependency can access, capability models help identify potentially unsafe or unexpected behavior in third-party code, a key concern in supply chain security.

4 Overview of planned work

- Research relevant background information about capabilities and capability models.
- Gather list of top 50 Golang packages.
- Analyze each package individually using `capslock`.
- Summarize results from each package to draw conclusions.
- Write project report.
- Attend the project presentation

5 Schedule

- **Week 17-18**: Background research and tooling setup. Verify `capslock` works as expected on a small sample of packages.
- **Week 19**: Collect the list of top 50 packages and automate the data collection pipeline.
- **Week 20**: Run full analysis across all packages and revisions.
- **Week 21-22**: Summarize results, draw conclusions, prepare presentation and write the report.
- **Week 23**: Final review and submission.

6 Target grade

The target grade is C-A. If the analysis using capslock can be considered as a practical implementation (it can be argued that it is a practical implementation for analysis), then higher grades should be awarded (A or B).

[1] `https://en.wikipedia.org/wiki/Capaility-based_security`
[2] `https://security.googleblog.com/2023/09/capslock-what-is-your-code-really.html`
[3] `https://github.com/google/capslock`